

PRACTICA – SEMANA 07

INTEGRANTES:

EDGAR GENARO PARI BENITES

YANILKA CARLA CASTRO DELGADO

Ejercicio 1. Boleta simple con IGV usando funciones

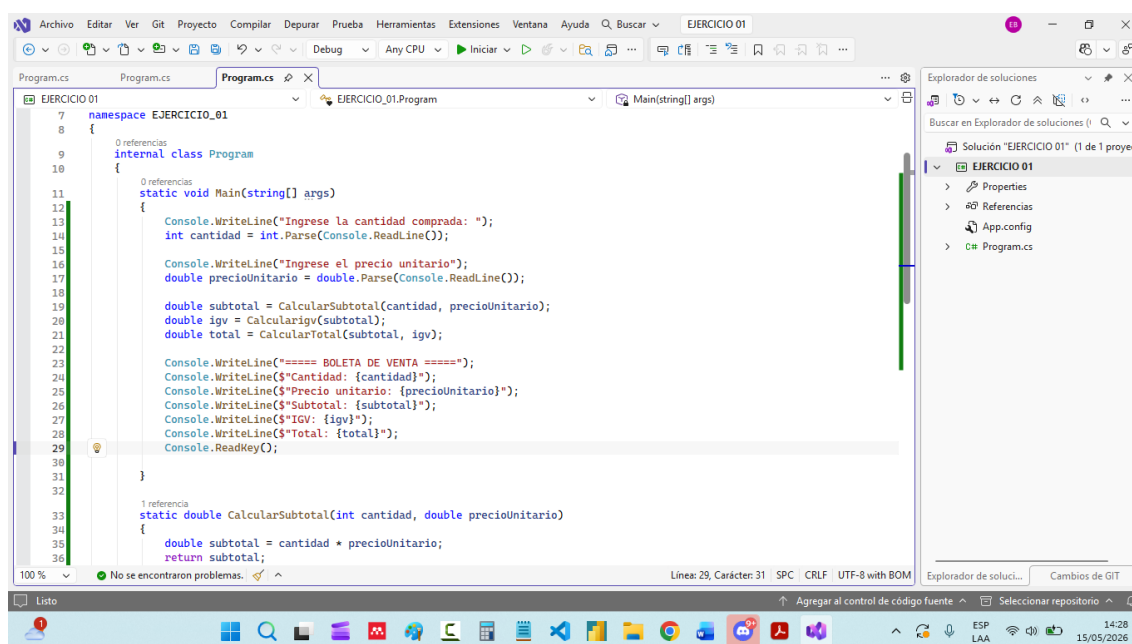
Tipo: Funciones con retorno y parámetros.

Una tienda vende un producto. El programa debe leer precio unitario y cantidad, calcular subtotal, IGV 18 % y total. Se usará una función para cada cálculo y otra función para mostrar la boleta.

Casos de prueba

Caso	Entrada	Salida esperada
1	precio=50, cantidad=2	Subtotal S/100.00; IGV S/18.00; Total S/118.00
2	precio=25.50, cantidad=4	Subtotal S/102.00; IGV S/18.36; Total S/120.36

EN C#



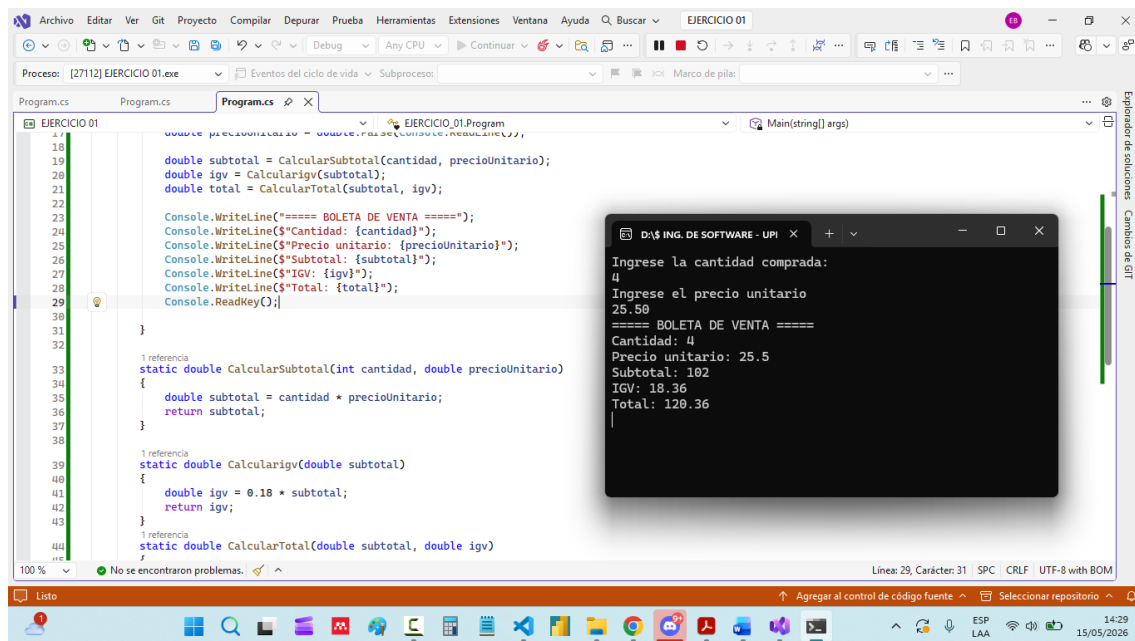
```
7 namespace EJERCICIO_01
8 {
9     internal class Program
10    {
11        0 referencias
12        static void Main(string[] args)
13        {
14            Console.WriteLine("Ingrese la cantidad comprada: ");
15            int cantidad = int.Parse(Console.ReadLine());
16
17            Console.WriteLine("Ingrese el precio unitario");
18            double precioUnitario = double.Parse(Console.ReadLine());
19
20            double subtotal = CalcularSubtotal(cantidad, precioUnitario);
21            double igv = Calcularigv(subtotal);
22            double total = CalcularTotal(subtotal, igv);
23
24            Console.WriteLine("==== BOLETA DE VENTA =====");
25            Console.WriteLine($"Cantidad: {cantidad}");
26            Console.WriteLine($"Precio unitario: {precioUnitario}");
27            Console.WriteLine($"Subtotal: {subtotal}");
28            Console.WriteLine($"IGV: {igv}");
29            Console.WriteLine($"Total: {total}");
30            Console.ReadKey();
31        }
32    }
33
34    1 referencia
35    static double CalcularSubtotal(int cantidad, double precioUnitario)
36    {
37        double subtotal = cantidad * precioUnitario;
38        return subtotal;
39    }
40
41    static double Calcularigv(double subtotal)
42    {
43        double igv = subtotal * 0.18;
44        return igv;
45    }
46
47    static double CalcularTotal(double subtotal, double igv)
48    {
49        double total = subtotal + igv;
50        return total;
51    }
52 }
```

```
23 Console.WriteLine("===== BOLETA DE VENTA =====");
24 Console.WriteLine($"Cantidad: {cantidad}");
25 Console.WriteLine($"Precio unitario: {precioUnitario}");
26 Console.WriteLine($"Subtotal: {subtotal}");
27 Console.WriteLine($"IGV: {igv}");
28 Console.WriteLine($"Total: {total}");
29 Console.ReadKey();
30
31 }
32
33 1 referencia
34 static double CalcularSubtotal(int cantidad, double precioUnitario)
35 {
36     double subtotal = cantidad * precioUnitario;
37     return subtotal;
38 }
39
40 1 referencia
41 static double CalcularIgv(double subtotal)
42 {
43     double igv = 0.18 * subtotal;
44     return igv;
45 }
46
47 1 referencia
48 static double CalcularTotal(double subtotal, double igv)
49 {
50     double total = subtotal + igv;
51     return total;
52 }
```

```
18 double precioUnitario = double.Parse(Console.ReadLine());
19
20 double subtotal = CalcularSubtotal(cantidad, precioUnitario);
21 double igv = CalcularIgv(subtotal);
22 double total = CalcularTotal(subtotal, igv);
23
24 Console.WriteLine("===== BOLETA DE VENTA =====");
25 Console.WriteLine($"Cantidad: {cantidad}");
26 Console.WriteLine($"Precio unitario: {precioUnitario}");
27 Console.WriteLine($"Subtotal: {subtotal}");
28 Console.WriteLine($"IGV: {igv}");
29 Console.WriteLine($"Total: {total}");
30 Console.ReadKey();
31
32 }
33
34 1 referencia
35 static double CalcularSubtotal(int cantidad, double precioUnitario)
36 {
37     double subtotal = cantidad * precioUnitario;
38     return subtotal;
39 }
40
41 1 referencia
42 static double CalcularIgv(double subtotal)
43 {
44     double igv = 0.18 * subtotal;
45     return igv;
46 }
47
48 1 referencia
49 static double CalcularTotal(double subtotal, double igv)
50 {
51     double total = subtotal + igv;
52     return total;
53 }
```

D:\\$ ING. DE SOFTWARE

Ingrese la cantidad comprada:
2
Ingrese el precio unitario
50
===== BOLETA DE VENTA =====
Cantidad: 2
Precio unitario: 50
Subtotal: 100
IGV: 18
Total: 118



Ejercicio 2. Reporte de notas con funciones

Tipo: Funciones con retorno, procedimiento y variable global/estática.

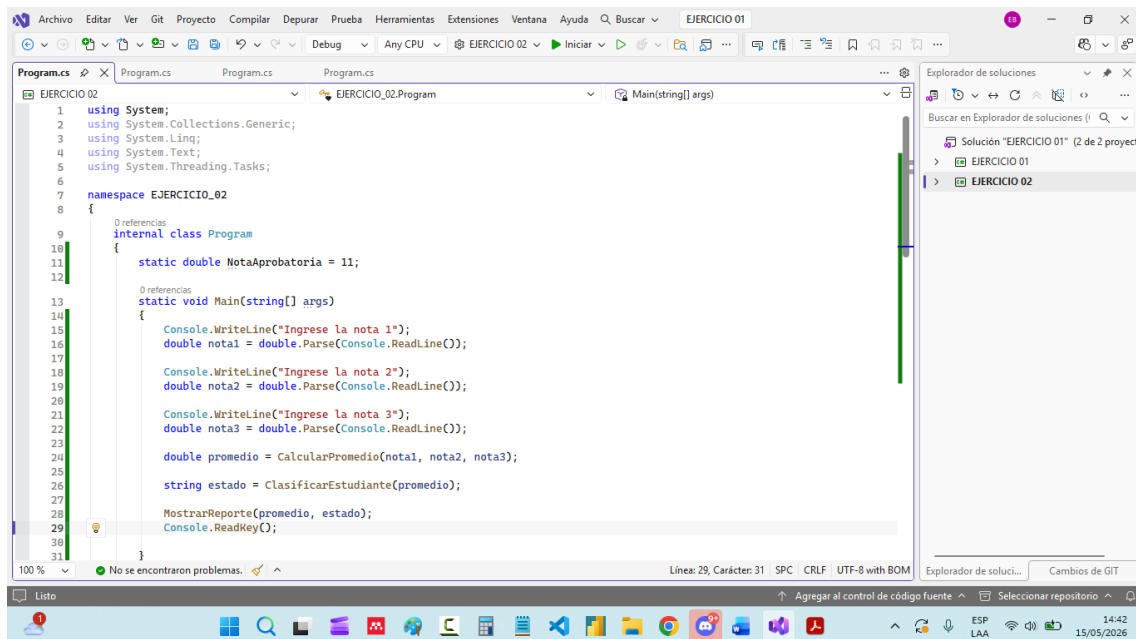
Un docente ingresa tres notas de un estudiante en escala de 0 a 20. El programa debe calcular el promedio, determinar si el estudiante está Aprobado o Desaprobado, y mostrar un reporte final.

La nota mínima aprobatoria es 11 y debe declararse como una variable global/estática para que pueda ser usada por la función de clasificación.

El programa debe usar:

- una función con retorno para calcular el promedio;
- una función con retorno para clasificar al estudiante;
- una función void en C# o procedimiento en Python para mostrar el reporte.

EN C#



The screenshot shows the Visual Studio IDE with a C# project named 'EJERCICIO 02'. The code is as follows:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_02
8 {
9     0 referencias
10     internal class Program
11     {
12         static double NotaAprobatoria = 11;
13
14         0 referencias
15         static void Main(string[] args)
16         {
17             Console.WriteLine("Ingrese la nota 1");
18             double nota1 = double.Parse(Console.ReadLine());
19
20             Console.WriteLine("Ingrese la nota 2");
21             double nota2 = double.Parse(Console.ReadLine());
22
23             Console.WriteLine("Ingrese la nota 3");
24             double nota3 = double.Parse(Console.ReadLine());
25
26             double promedio = CalcularPromedio(nota1, nota2, nota3);
27
28             string estado = ClasificarEstudiante(promedio);
29
30             MostrarReporte(promedio, estado);
31             Console.ReadKey();
32         }
33     }
34 }
```

The interface also shows the 'Explorador de soluciones' (Solution Explorer) on the right, displaying the project structure with 'EJERCICIO 01' and 'EJERCICIO 02'. The status bar at the bottom indicates 'Linea: 29, Carácter: 31' and 'UTF-8 with BOM'.

```
21 Console.WriteLine("Ingrese la nota 3");
22 double nota3 = double.Parse(Console.ReadLine());
23
24 double promedio = CalcularPromedio(nota1, nota2, nota3);
25
26 string estado = ClasificarEstudiante(promedio);
27
28 MostrarReporte(promedio, estado);
29 Console.ReadKey();
30
31 }
32
33 1 referencia
34 static double CalcularPromedio(double nota1, double nota2, double nota3)
35 {
36     double promedio = (nota1 + nota2 + nota3) / 3;
37     return promedio;
38 }
39
40 1 referencia
41 static string ClasificarEstudiante(double promedio)
42 {
43     if (promedio >= NotaAprobatoria)
44     {
45         return "Aprobado";
46     }
47     else
48     {
49         return "Desaprobado";
50     }
51 }
52
53 1 referencia
54 static void MostrarReporte(double promedio, string estado)
```

```
55
56 }
57
58 }
59
60
61 1 referencia
62 static double CalcularPromedio(double nota1, double nota2, double nota3)
63 {
64     double promedio = (nota1 + nota2 + nota3) / 3;
65     return promedio;
66 }
67
68 1 referencia
69 static string ClasificarEstudiante(double promedio)
70 {
71     if (promedio >= NotaAprobatoria)
72     {
73         return "Aprobado";
74     }
75     else
76     {
77         return "Desaprobado";
78     }
79 }
80
81 1 referencia
82 static void MostrarReporte(double promedio, string estado)
83 {
84     Console.WriteLine("==== REPORTE DE NOTAS =====");
85     Console.WriteLine($"Promedio: {promedio:F2}");
86     Console.WriteLine($"Estado: {estado}");
87 }
88
89 }
```

Visual Studio interface showing the execution of a C# program. The code defines three methods: `CalcularPromedio`, `ClasificarEstudiante`, and `MostrarReporte`. The console output shows the following results:

```
Ingresa la nota 1
15
Ingresa la nota 2
18
Ingresa la nota 3
12
===== REPORTE DE NOTAS =====
Promedio: 15.00
Estado: Aprobado
```

Visual Studio interface showing the execution of the same C# program with different input values. The console output shows the following results:

```
Ingresa la nota 1
8
Ingresa la nota 2
10
Ingresa la nota 3
11
===== REPORTE DE NOTAS =====
Promedio: 9.67
Estado: Desaprobado
```

Ejercicio 3. Calculadora de sueldo neto

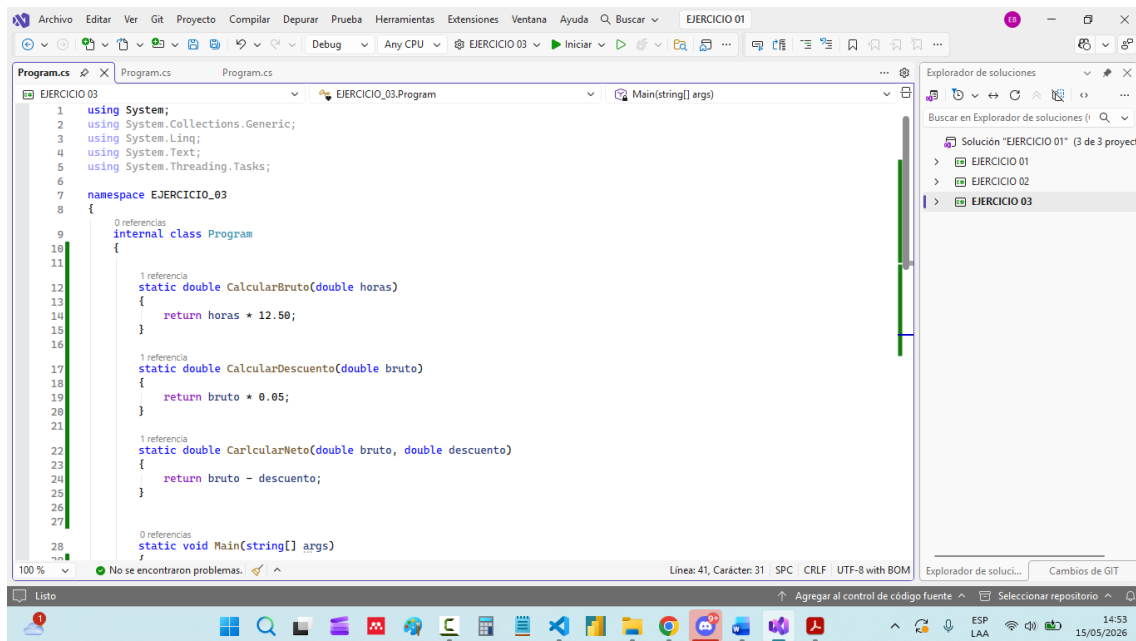
Tipo: Funciones con retorno, parámetros y operaciones aritméticas.

Una empresa paga S/12.50 por hora trabajada. El programa debe calcular:

- sueldo bruto;
- descuento del 5 %;
- sueldo neto.

Cada cálculo debe realizarse usando una función con retorno.

EN C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_03
8 {
9     0 referencias
10     internal class Program
11     {
12         1 referencia
13         static double CalcularBruto(double horas)
14         {
15             return horas * 12.50;
16         }
17
18         1 referencia
19         static double CalcularDescuento(double bruto)
20         {
21             return bruto * 0.05;
22         }
23
24         1 referencia
25         static double CalcularNeto(double bruto, double descuento)
26         {
27             return bruto - descuento;
28         }
29
30         0 referencias
31         static void Main(string[] args)
32         {
33         }
34     }
35 }
```

```
17 static double CalcularDescuento(double bruto)
18 {
19     return bruto * 0.05;
20 }
21
22 1 referencia
23 static double CarcularNeto(double bruto, double descuento)
24 {
25     return bruto - descuento;
26 }
27
28 0 referencias
29 static void Main(string[] args)
30 {
31     Console.WriteLine("Diga las horas trabajadas: ");
32     double horas = double.Parse(Console.ReadLine());
33
34     double bruto = CalcularBruto(horas);
35     double descuento = CalcularDescuento(bruto);
36     double neto = CarcularNeto(bruto, descuento);
37
38     Console.WriteLine("==== BOLETA DE SUELDO =====");
39     Console.WriteLine($"Sueldo bruto: {bruto:F2}");
40     Console.WriteLine($"Descuento 5%: {descuento:F2}");
41     Console.WriteLine($"Sueldo neto: {neto:F2}");
42     Console.ReadKey();
43 }
44
45 }
46 }
```

100 % No se encontraron problemas. Línea: 41, Carácter: 31 SPC CRLF UTF-8 with BOM

Explorador de soluciones

Solución "EJERCICIO 01" (3 de 3 proyectos)

- EJERCICIO 01
- EJERCICIO 02
- EJERCICIO 03

14:54 15/05/2026

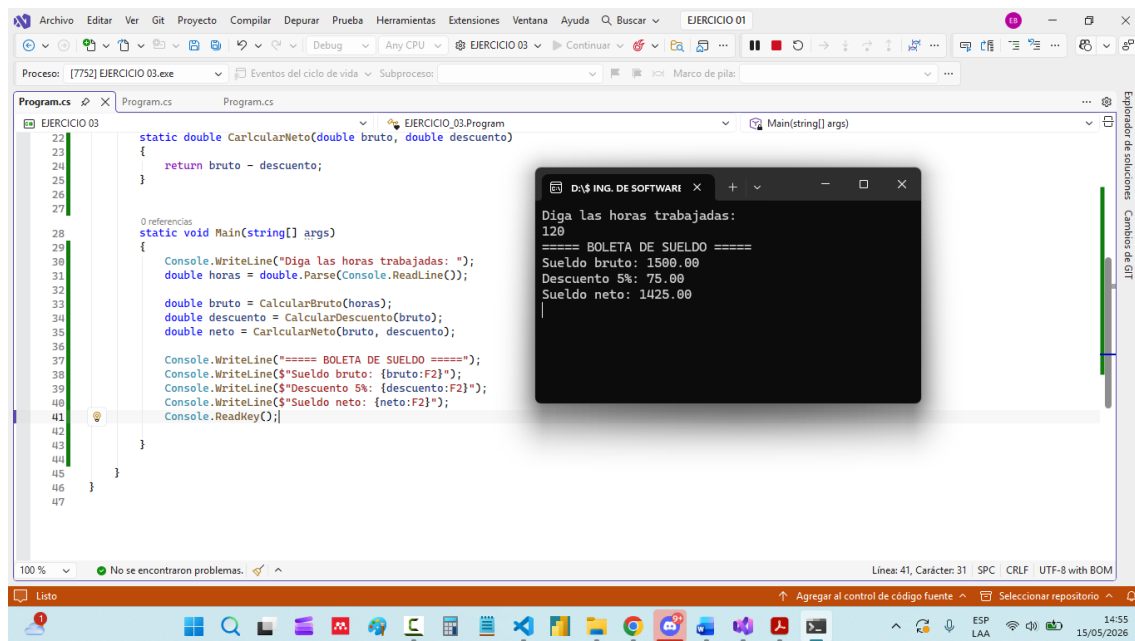
```
22 static double CarcularNeto(double bruto, double descuento)
23 {
24     return bruto - descuento;
25 }
26
27
28 0 referencias
29 static void Main(string[] args)
30 {
31     Console.WriteLine("Diga las horas trabajadas: ");
32     double horas = double.Parse(Console.ReadLine());
33
34     double bruto = CalcularBruto(horas);
35     double descuento = CalcularDescuento(bruto);
36     double neto = CarcularNeto(bruto, descuento);
37
38     Console.WriteLine("==== BOLETA DE SUELDO =====");
39     Console.WriteLine($"Sueldo bruto: {bruto:F2}");
40     Console.WriteLine($"Descuento 5%: {descuento:F2}");
41     Console.WriteLine($"Sueldo neto: {neto:F2}");
42     Console.ReadKey();
43 }
44
45 }
46 }
47 }
```

100 % No se encontraron problemas. Línea: 41, Carácter: 31 SPC CRLF UTF-8 with BOM

Proceso: [22356] EJERCICIO_03.exe

Explorador de soluciones

14:54 15/05/2026



Ejercicio 4. Casa de cambio con comisión

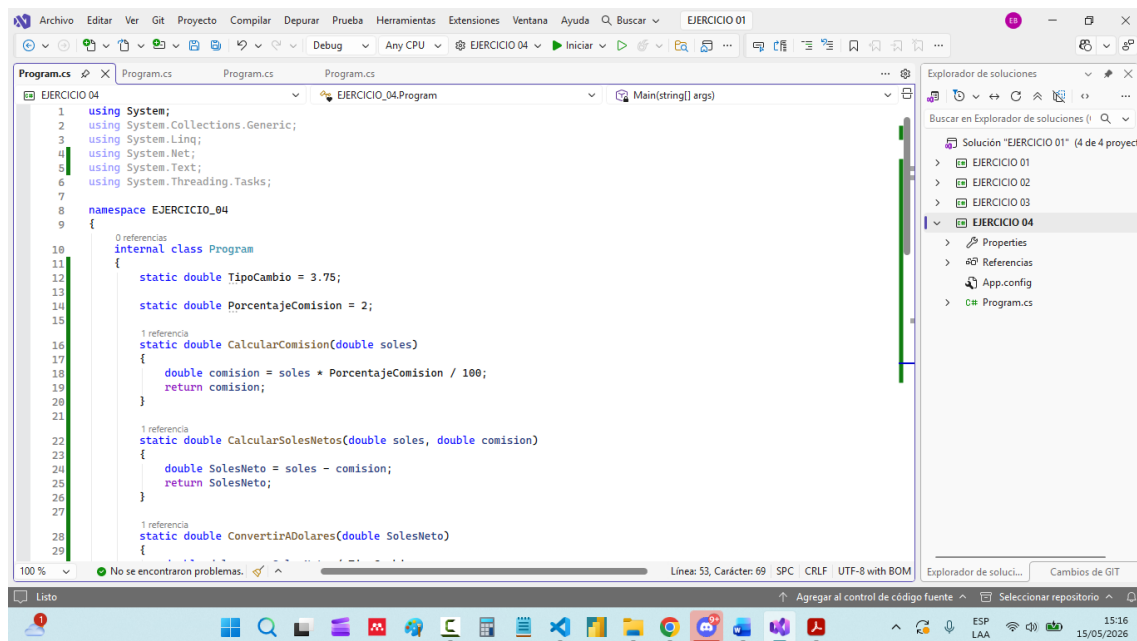
Tipo: Funciones con retorno, parámetros, procedimiento y operaciones aritméticas.

Una casa de cambio convierte soles a dólares usando un tipo de cambio de **S/3.75 por dólar**. Además, cobra una comisión del **2 %** sobre el monto en soles cambiado.

El programa debe solicitar el monto en soles y, usando funciones, calcular:

- la comisión;
- el monto neto en soles después de descontar la comisión;
- los dólares que recibirá el cliente;
- mostrar un comprobante final.

EN C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Net;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace EJERCICIO_04
9 {
10     internal class Program
11     {
12         static double TipoCambio = 3.75;
13
14         static double PorcentajeComision = 2;
15
16         static double CalcularComision(double soles)
17         {
18             double comision = soles * PorcentajeComision / 100;
19             return comision;
20         }
21
22         static double CalcularSolesNetos(double soles, double comision)
23         {
24             double SolesNeto = soles - comision;
25             return SolesNeto;
26         }
27
28         static double ConvertirADolares(double SolesNeto)
29         {
30             // ...
31         }
32     }
33 }
```

```
23 {
24     double SolesNeto = soles - comision;
25     return SolesNeto;
26 }
27
28 1 referencia
29 static double ConvertirADolares(double SolesNeto)
30 {
31     double dolares = SolesNeto / TipoCambio;
32     return dolares;
33 }
34
35 1 referencia
36 static void MostrarComprobante(double soles, double comision, double SolesNeto, double dolares)
37 {
38     Console.WriteLine("==== BOLETA =====");
39     Console.WriteLine($"Monto ingresado: {soles:F2}");
40     Console.WriteLine($"Comision 2%: {comision:F2}");
41     Console.WriteLine($"Soles neto a cambiar: {SolesNeto:F2}");
42     Console.WriteLine($"Dolares recibidos: {dolares:F2}");
43     Console.ReadKey();
44 }
45
46 0 referencias
47 static void Main(string[] args)
48 {
49     Console.WriteLine("CASA DE CAMBIO");
50     Console.WriteLine("Ingrese el monto en soles: ");
51     double soles = double.Parse( Console.ReadLine() );
52
53     double comision = CalcularComision(soles);
54     double SolesNeto = CalcularSolesNetos(soles, comision);
55     double dolares = ConvertirADolares(SolesNeto);
```

```
31 }
32
33
34 1 referencia
35 static void MostrarComprobante(double soles, double comision, double SolesNeto, double dolares)
36 {
37     Console.WriteLine("==== BOLETA =====");
38     Console.WriteLine($"Monto ingresado: {soles:F2}");
39     Console.WriteLine($"Comision 2%: {comision:F2}");
40     Console.WriteLine($"Soles neto a cambiar: {SolesNeto:F2}");
41     Console.WriteLine($"Dolares recibidos: {dolares:F2}");
42     Console.ReadKey();
43 }
44
45 0 referencias
46 static void Main(string[] args)
47 {
48     Console.WriteLine("CASA DE CAMBIO");
49     Console.WriteLine("Ingrese el monto en soles: ");
50     double soles = double.Parse( Console.ReadLine() );
51
52     double comision = CalcularComision(soles);
53     double SolesNeto = CalcularSolesNetos(soles, comision);
54     double dolares = ConvertirADolares(SolesNeto);
55     MostrarComprobante(soles, comision, SolesNeto, dolares);
56 }
57
58 }
59 }
```

Archivo Editar Ver Git Proyecto Compilar Depurar Prueba Herramientas Extensiones Ventana Ayuda Q Buscar EJERCICIO 01

Proceso: [27256] EJERCICIO_04.exe

Program.cs

```
29 {
30     double dolares = SolesNeto / TipoCambio;
31     return dolares;
32 }
33
34 1 referencia
35 static void MostrarComprobante(double soles, double comision, double SolesNeto)
36 {
37     Console.WriteLine("==== BOLETA =====");
38     Console.WriteLine($"Monto ingresado: {soles:F2}");
39     Console.WriteLine($"Comision 2%: {comision:F2}");
40     Console.WriteLine($"Soles neto a cambiar: {SolesNeto:F2}");
41     Console.WriteLine($"Dolares recibidos: {dolares:F2}");
42     Console.ReadKey();
43 }
44
45 0 referencias
46 static void Main(string[] args)
47 {
48     Console.WriteLine("CASA DE CAMBIO");
49     Console.WriteLine("Ingrese el monto en soles: ");
50     double soles = double.Parse( Console.ReadLine() );
51
52     double comision = CalcularComision(soles);
53     double SolesNeto = CalcularSolesNetos(soles, comision);
54     double dolares = ConvertirADolares(SolesNeto);
55     MostrarComprobante(soles, comision, SolesNeto, dolares);
56 }
57 }
58
59 100 % No se encontraron problemas. Línea: 53, Carácter: 69 SPC CRLF UTF-8 with BOM

D:\$ ING. DE SOFTWARE



CASA DE CAMBIO  
Ingrese el monto en soles:  
375  
==== BOLETA =====  
Monto ingresado: 375.00  
Comision 2%: 7.50  
Soles neto a cambiar: 367.50  
Dolares recibidos: 98



Listo Agregar al control de código fuente Seleccionar repositorio 15:17 ESP LAA 15/05/2026


```

Archivo Editar Ver Git Proyecto Compilar Depurar Prueba Herramientas Extensiones Ventana Ayuda Q Buscar EJERCICIO 01

Proceso: [30532] EJERCICIO_04.exe

Program.cs

```
34 static void MostrarComprobante(double soles, double comision, double SolesNeto, double dolares)
35 {
36     Console.WriteLine("==== BOLETA =====");
37     Console.WriteLine($"Monto ingresado: {soles:F2}");
38     Console.WriteLine($"Comision 2%: {comision:F2}");
39     Console.WriteLine($"Soles neto a cambiar: {SolesNeto:F2}");
40     Console.WriteLine($"Dolares recibidos: {dolares:F2}");
41     Console.ReadKey();
42 }
43
44 0 referencias
45 static void Main(string[] args)
46 {
47     Console.WriteLine("CASA DE CAMBIO");
48     Console.WriteLine("Ingrese el monto en soles: ");
49     double soles = double.Parse( Console.ReadLine() );
50
51     double comision = CalcularComision(soles);
52     double SolesNeto = CalcularSolesNetos(soles, comision);
53     double dolares = ConvertirADolares(SolesNeto);
54     MostrarComprobante(soles, comision, SolesNeto, dolares);
55 }
56 }
57 }
58 }
59 }
60
61 100 % No se encontraron problemas. Línea: 53, Carácter: 69 SPC CRLF UTF-8 with BOM

D:\$ ING. DE SOFTWARE

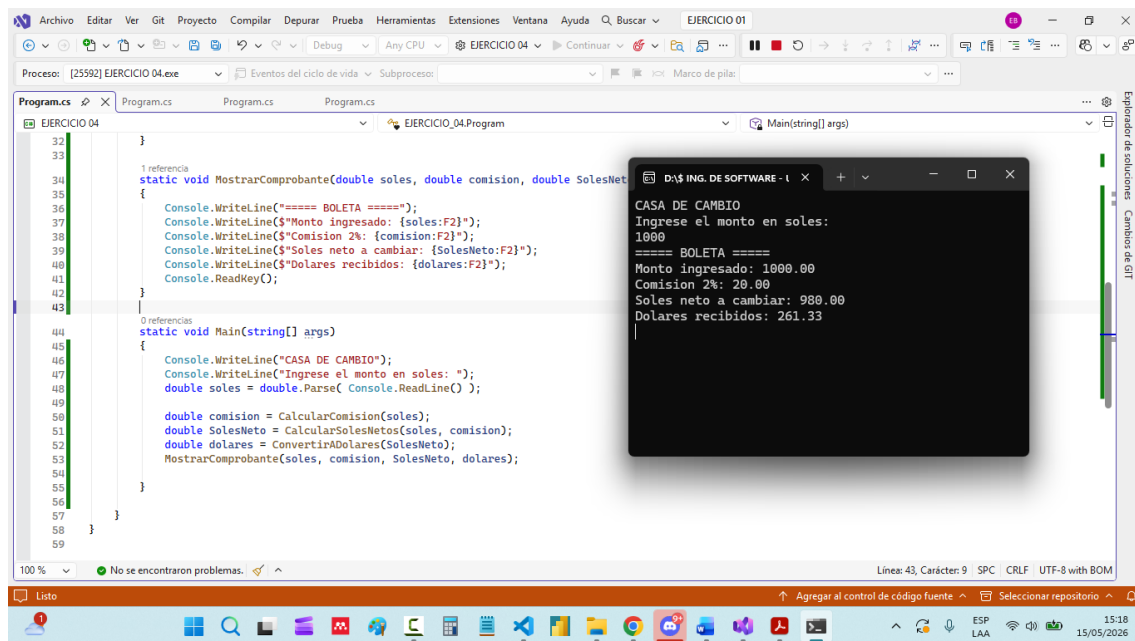


CASA DE CAMBIO  
Ingrese el monto en soles:  
750  
==== BOLETA =====  
Monto ingresado: 750.00  
Comision 2%: 15.00  
Soles neto a cambiar: 735.00  
Dolares recibidos: 196



Listo Agregar al control de código fuente Seleccionar repositorio 15:17 ESP LAA 15/05/2026


```



Ejercicio 5. Clasificador de nota con validación y reporte

Tipo: Funciones con retorno, parámetros, validación y procedimiento/void.

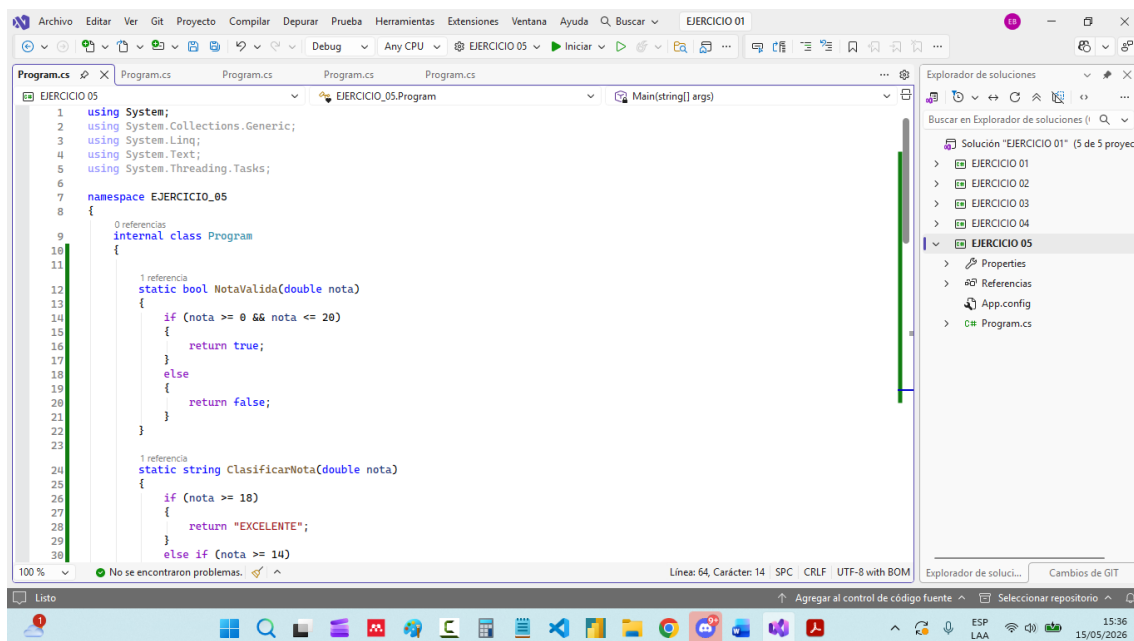
Un sistema académico necesita registrar la nota final de un estudiante en escala de 0 a 20. El programa debe validar que la nota esté dentro del rango permitido y luego clasificarla como:

Rango	Clasificación
18 - 20	Excelente
14 - 17	Bueno
11 - 13	Regular
0 - 10	Desaprobado

El programa debe usar:

- una función con retorno para validar si la nota es correcta;
- una función con retorno para clasificar la nota;
- una función void en C# para mostrar el reporte.

EN C#



The screenshot shows the Visual Studio IDE with a C# project named 'EJERCICIO_05'. The code is as follows:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_05
8 {
9     0 referencias
10     internal class Program
11     {
12         1 referencia
13         static bool NotaValida(double nota)
14         {
15             if (nota >= 0 && nota <= 20)
16             {
17                 return true;
18             }
19             else
20             {
21                 return false;
22             }
23         }
24
25         1 referencia
26         static string ClasificarNota(double nota)
27         {
28             if (nota >= 18)
29             {
30                 return "EXCELENTE";
31             }
32             else if (nota >= 14)
33             {
34                 return "BUENO";
35             }
36             else if (nota >= 11)
37             {
38                 return "REGULAR";
39             }
40             else
41             {
42                 return "DESAPROBADO";
43             }
44         }
45     }
46 }
```

The right sidebar shows the 'Explorador de soluciones' (Solution Explorer) with the project structure:

- Solución "EJERCICIO 01" (5 de 3 proyectos)
 - EJERCICIO 01
 - EJERCICIO 02
 - EJERCICIO 03
 - EJERCICIO 04
 - EJERCICIO 05**
 - Properties
 - Referencias
 - App.config
 - Program.cs**

The status bar at the bottom indicates 'Listo' (Ready) and 'No se encontraron problemas.' (No problems found).

This screenshot shows the Visual Studio IDE with the 'Program.cs' file open. The code defines a static method `ClasificarNota(double nota)` that returns a string classification based on the input note. The classification logic is as follows:

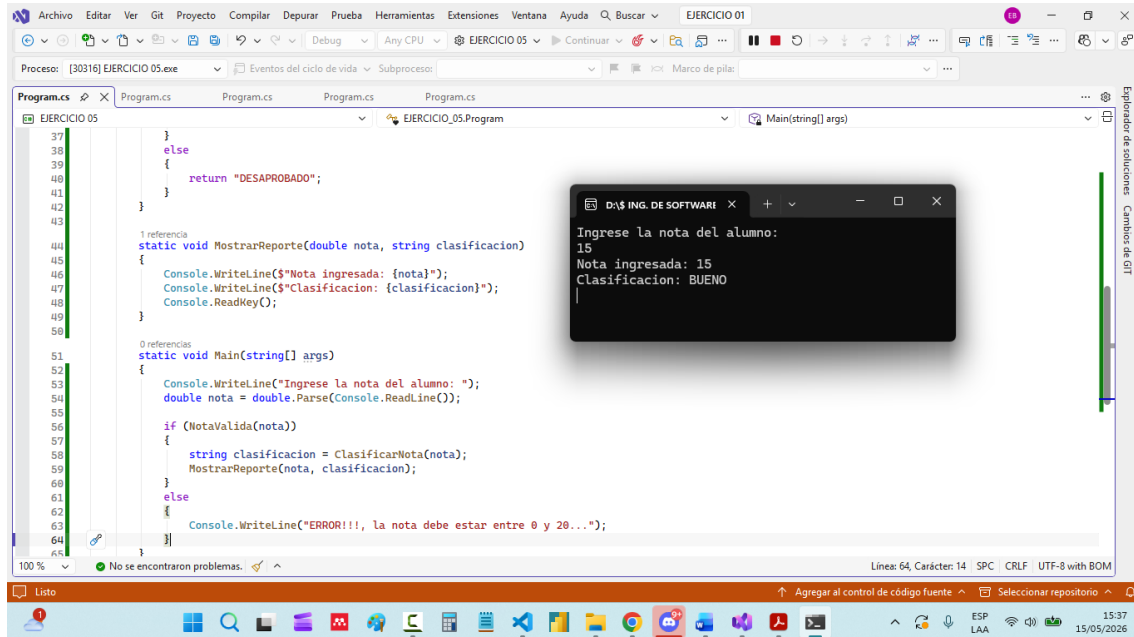
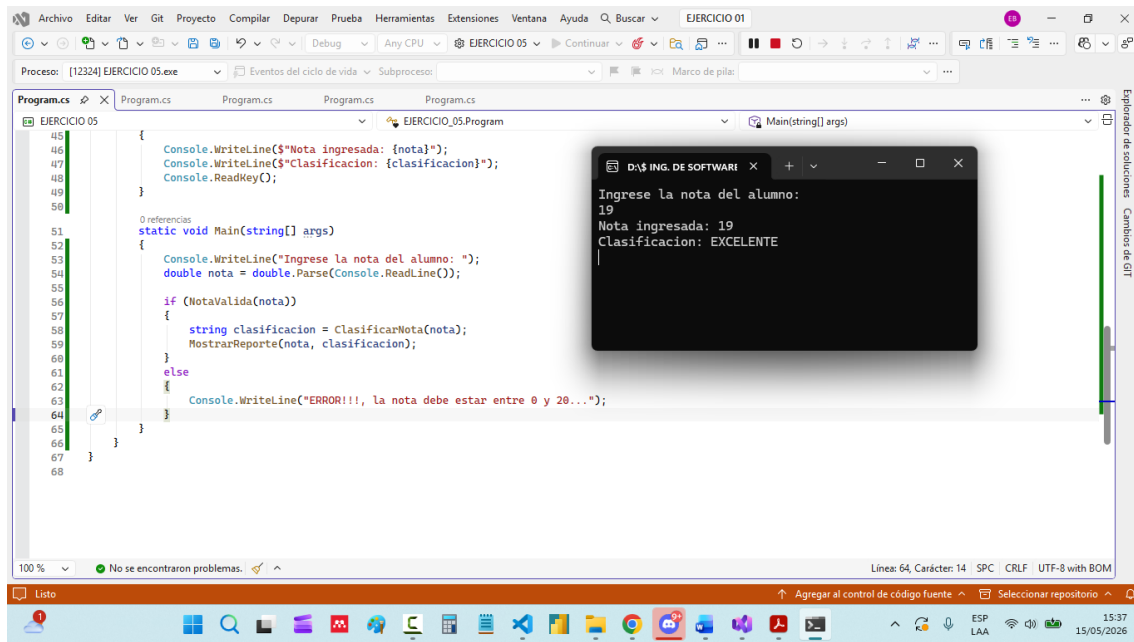
- If `nota >= 18`, return `"EXCELENTE"`.
- Else if `nota >= 14`, return `"BUENO"`.
- Else if `nota >= 11`, return `"REGULAR"`.
- Else, return `"DESAPROBADO"`.

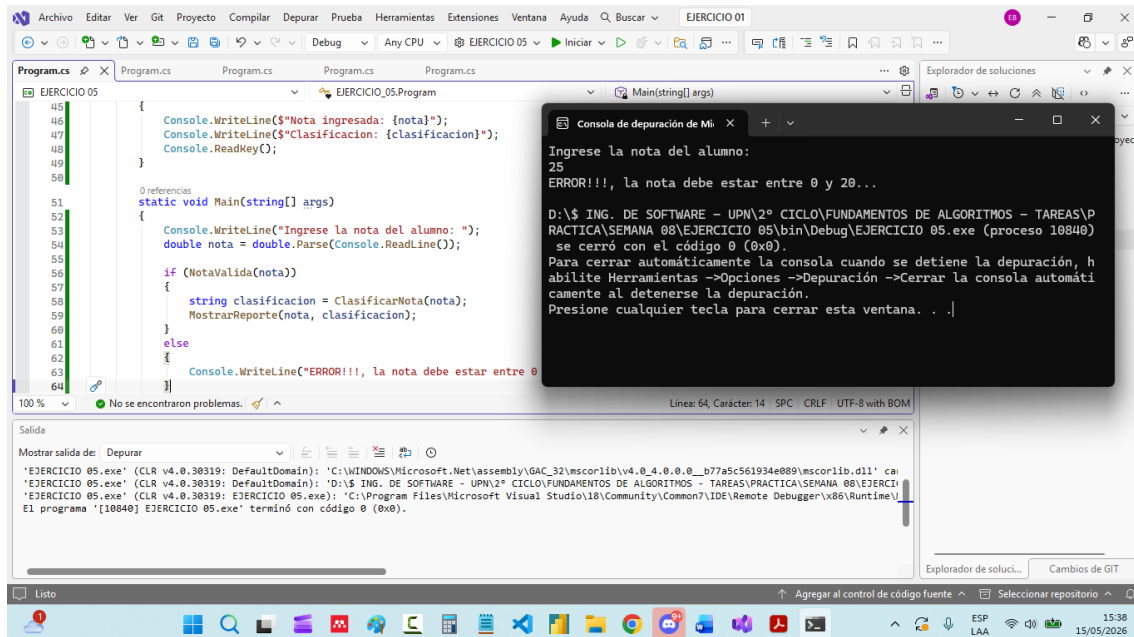
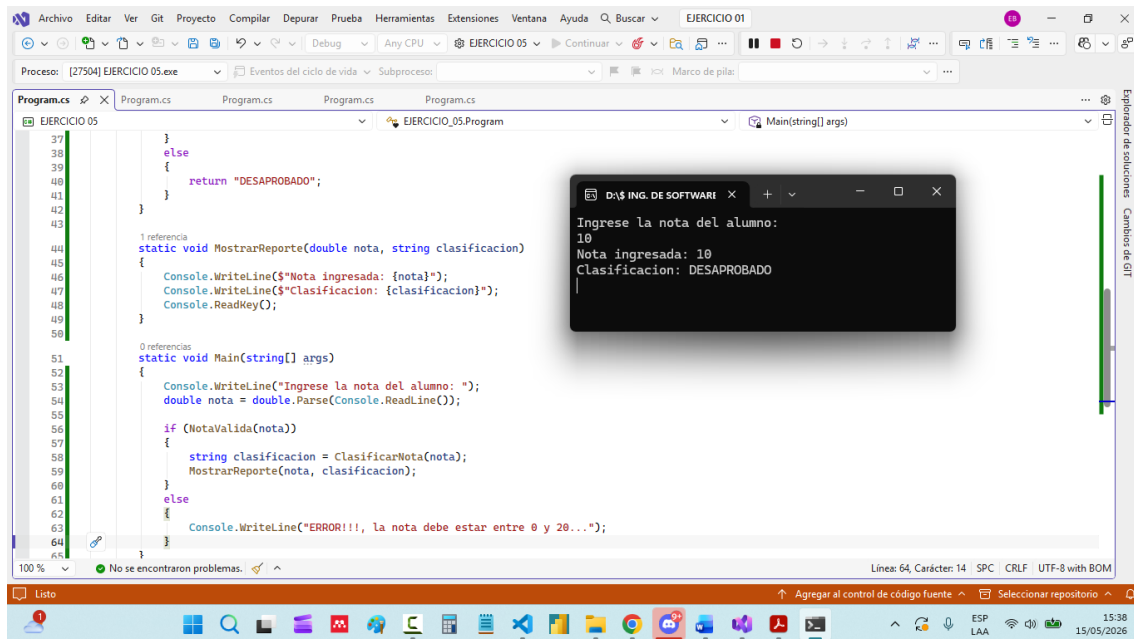
Below this method, there is a static method `MostrarReporte(double nota, string clasificacion)` that writes the input note and its classification to the console and reads a key from the console.

The right sidebar shows the 'Explorador de soluciones' (Solution Explorer) with the project structure: 'Solución "EJERCICIO 01"' (5 de 5 proyectos) containing 'EJERCICIO 01', 'EJERCICIO 02', 'EJERCICIO 03', 'EJERCICIO 04', and 'EJERCICIO 05'. 'EJERCICIO 05' is currently selected.

This screenshot shows the Visual Studio IDE with the 'Program.cs' file open, displaying the `Main` method. The `Main` method prompts the user to enter a note, parses it as a double, and then calls `ClasificarNota` to get the classification. It then calls `MostrarReporte` to display the results. If the note is invalid (not between 0 and 20), it prints an error message.

The right sidebar shows the 'Explorador de soluciones' (Solution Explorer) with the project structure: 'Solución "EJERCICIO 01"' (5 de 5 proyectos) containing 'EJERCICIO 01', 'EJERCICIO 02', 'EJERCICIO 03', 'EJERCICIO 04', and 'EJERCICIO 05'. 'EJERCICIO 05' is currently selected.





Ejercicio 6. Consumo de agua doméstico

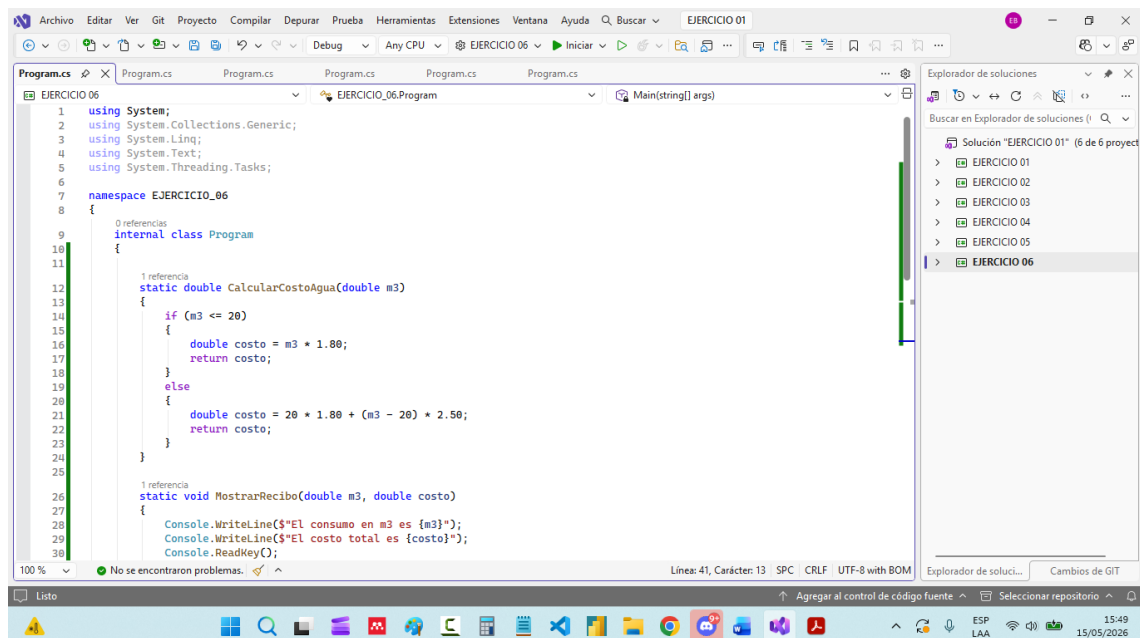
Tipo: Funciones con retorno, parámetros, condicional y procedimiento.

Una empresa de agua aplica una tarifa simplificada según el consumo mensual de una vivienda. Si el consumo es de hasta **20 m³**, se cobra **\$/1.80 por m³**. Si el consumo supera los **20 m³**, los primeros 20 m³ se cobran a **\$/1.80** y el exceso se cobra a **\$/2.50 por m³**.

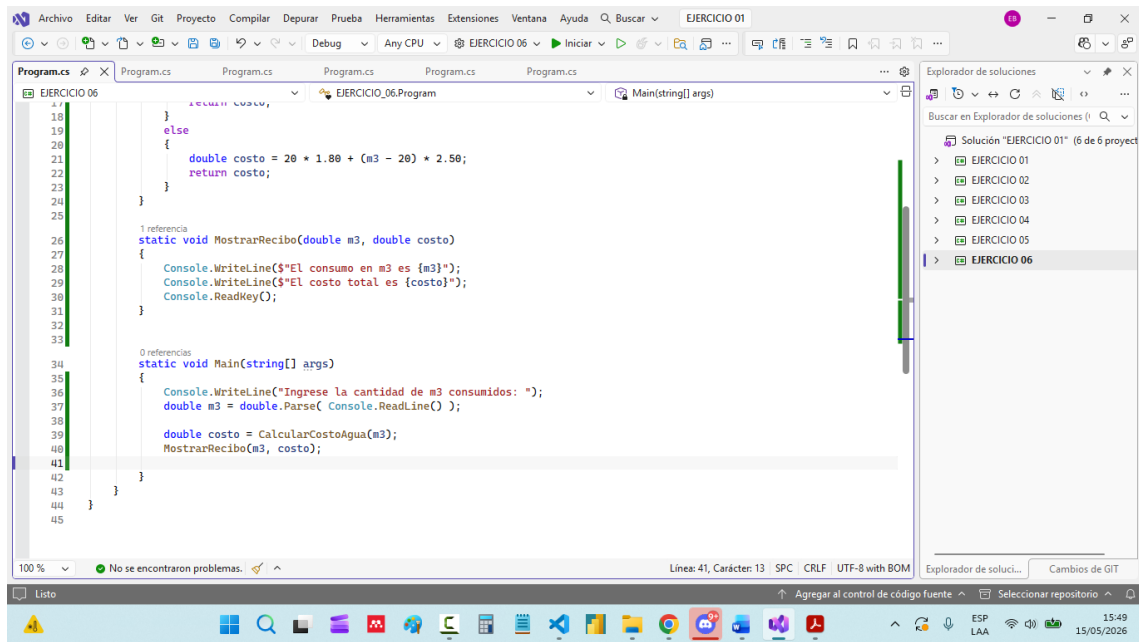
Crea un programa usando funciones para:

- calcular el costo del consumo;
- mostrar el recibo final con el consumo y el monto a pagar.

EN C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_06
8 {
9
10     internal class Program
11     {
12         1 referencia
13         static double CalcularCostoAgua(double m3)
14         {
15             if (m3 <= 20)
16             {
17                 double costo = m3 * 1.80;
18                 return costo;
19             }
20             else
21             {
22                 double costo = 20 * 1.80 + (m3 - 20) * 2.50;
23                 return costo;
24             }
25         }
26
27         1 referencia
28         static void MostrarRecibo(double m3, double costo)
29         {
30             Console.WriteLine($"El consumo en m3 es {m3}");
31             Console.WriteLine($"El costo total es {costo}");
32             Console.ReadKey();
33         }
34     }
35 }
```



```
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45
```

1 referencia

```
static void MostrarRecibo(double m3, double costo)  
{  
    Console.WriteLine($"El consumo en m3 es {m3}");  
    Console.WriteLine($"El costo total es {costo}");  
    Console.ReadKey();  
}
```

0 referencias

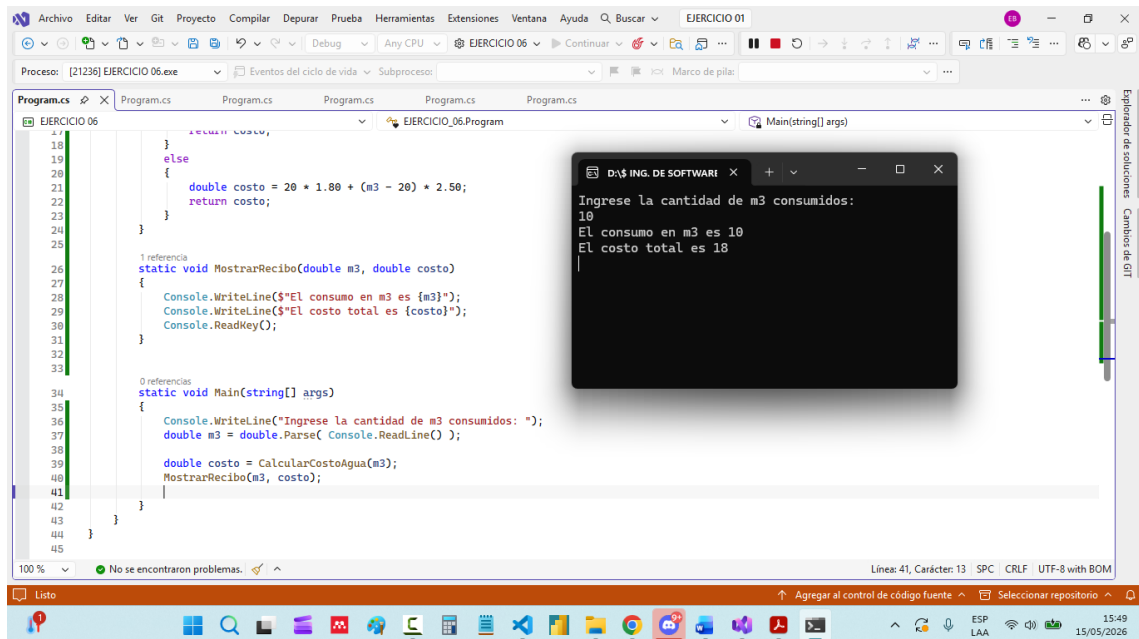
```
static void Main(string[] args)  
{  
    Console.WriteLine("Ingrese la cantidad de m3 consumidos: ");  
    double m3 = double.Parse( Console.ReadLine() );  
  
    double costo = CalcularCostoAgua(m3);  
    MostrarRecibo(m3, costo);  
}
```

100 % No se encontraron problemas. Línea: 41, Carácter: 13 SPC CRLF UTF-8 with BOM

Explorador de soluciones

Solución "EJERCICIO 01" (6 de 6 proyectos)

- EJERCICIO 01
- EJERCICIO 02
- EJERCICIO 03
- EJERCICIO 04
- EJERCICIO 05
- EJERCICIO 06



```
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45
```

1 referencia

```
static void MostrarRecibo(double m3, double costo)  
{  
    Console.WriteLine($"El consumo en m3 es {m3}");  
    Console.WriteLine($"El costo total es {costo}");  
    Console.ReadKey();  
}
```

0 referencias

```
static void Main(string[] args)  
{  
    Console.WriteLine("Ingrese la cantidad de m3 consumidos: ");  
    double m3 = double.Parse( Console.ReadLine() );  
  
    double costo = CalcularCostoAgua(m3);  
    MostrarRecibo(m3, costo);  
}
```

100 % No se encontraron problemas. Línea: 41, Carácter: 13 SPC CRLF UTF-8 with BOM

Explorador de soluciones

Solución "EJERCICIO 01" (6 de 6 proyectos)

- EJERCICIO 01
- EJERCICIO 02
- EJERCICIO 03
- EJERCICIO 04
- EJERCICIO 05
- EJERCICIO 06

Proceso: [21236] EJERCICIO 06.exe

Eventos del ciclo de vida Subprocesos:

Marco de pila:

D:\\$ ING. DE SOFTWARE

Ingrese la cantidad de m3 consumidos:
10
El consumo en m3 es 10
El costo total es 18

```
23 }
24
25
26 1 referencia
27 static void MostrarRecibo(double m3, double costo)
28 {
29     Console.WriteLine($"El consumo en m3 es {m3}");
30     Console.WriteLine($"El costo total es {costo}");
31     Console.ReadKey();
32 }
33
34 0 referencias
35 static void Main(string[] args)
36 {
37     Console.WriteLine("Ingrese la cantidad de m3 consumidos: ");
38     double m3 = double.Parse( Console.ReadLine() );
39
40     double costo = CalcularCostoAgua(m3);
41     MostrarRecibo(m3, costo);
42 }
43
44 }
45
```

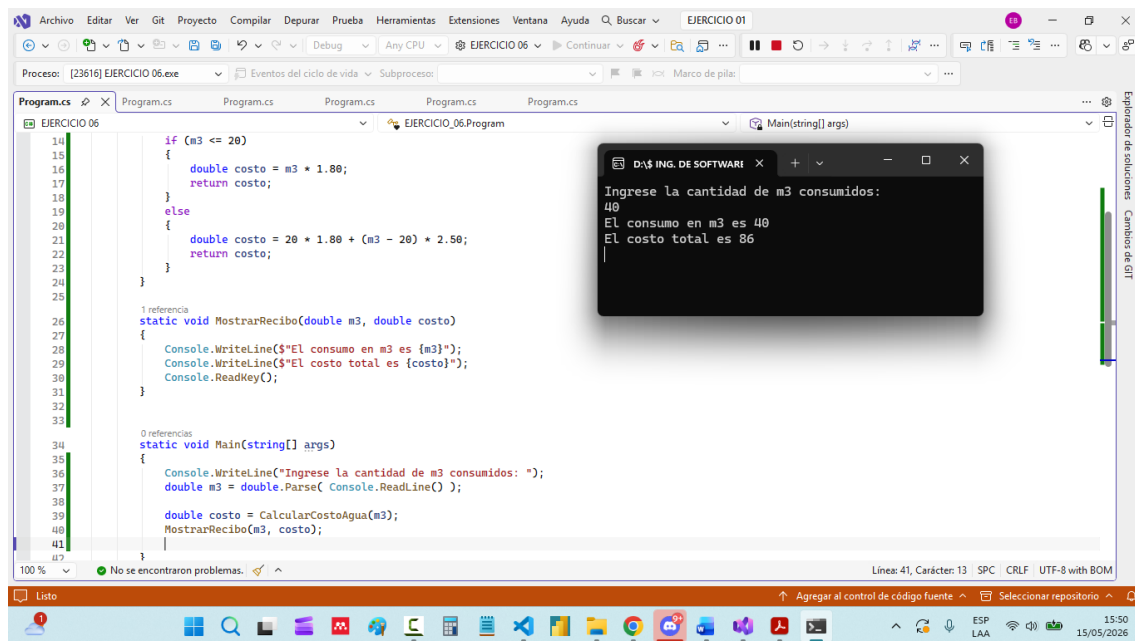
Console Output:

```
D:\$ ING. DE SOFTWARE >
Ingrese la cantidad de m3 consumidos:
20
El consumo en m3 es 20
El costo total es 36
```

```
17     return costo;
18 }
19 else
20 {
21     double costo = 20 * 1.80 + (m3 - 20) * 2.50;
22     return costo;
23 }
24
25
26 1 referencia
27 static void MostrarRecibo(double m3, double costo)
28 {
29     Console.WriteLine($"El consumo en m3 es {m3}");
30     Console.WriteLine($"El costo total es {costo}");
31     Console.ReadKey();
32 }
33
34 0 referencias
35 static void Main(string[] args)
36 {
37     Console.WriteLine("Ingrese la cantidad de m3 consumidos: ");
38     double m3 = double.Parse( Console.ReadLine() );
39
40     double costo = CalcularCostoAgua(m3);
41     MostrarRecibo(m3, costo);
42 }
43
44 }
45
```

Console Output:

```
D:\$ ING. DE SOFTWARE >
Ingrese la cantidad de m3 consumidos:
25
El consumo en m3 es 25
El costo total es 48.5
```



COMANDOS:

git init

git add .

git remote add origin "----" enlace de github

git commit -m "PRACTICA - TAREA - SEMANA 08"

git pull origin master

..... si hay problema entonces se pone

git pull origin master --allow-unrelated-histories

git push -u origin master